

Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Звіт до комп’ютерного практикуму №3 з дисципліни «Комп’ютерне моделювання дискретно-подійних систем»

Прийняв:
Викладач
Д.С. Коринецький

Виконав
Студент 4 курсу, гр. ІП-33
Ломакін Максим

Оцінка: _____

«___» _____ 2026 р.

Київ 2026

Побудова імітаційної моделі системи з використанням формалізму моделі масового обслуговування

1. Текст завдання:

1. Реалізувати універсальний алгоритм імітації моделі масового обслуговування з багатоканальним обслуговуванням, з вибором маршруту за пріоритетом або за заданою ймовірністю. 30 балів.
2. Для наступного тексту задачі скласти формалізовану модель масового обслуговування та реалізувати її з використанням побудованого універсального алгоритму (30 балів):

У банку для автомобілістів є два віконця, кожне з яких обслуговується одним касиром і має окрему під'їзну смугу. Обидві смуги розташовані поруч. З попередніх спостережень відомо, що інтервали часу між прибуттям клієнтів у годину пік розподілені експоненційно з математичним сподіванням, рівним 0,5 од. часу. Через те, що банк буває переобтяжений тільки в годину пік, то аналізується тільки цей період. Тривалість обслуговування в обох касирів однакова і розподілена експоненційно з математичним сподіванням, рівним 0,3 од. часу. Відомо також, що при рівній довжині черг, а також при відсутності черг, клієнти віддають перевагу першій смузі. В усіх інших випадках клієнти вибирають більш коротку чергу. Після того, як клієнт в'їхав у банк, він не може залишити його, доки не буде обслугований. Проте він може змінити чергу, якщо стоїть останнім і різниця в довжині черг при цьому складає не менше двох автомобілів. Через обмежене місце на кожній смузі може знаходитися не більш трьох автомобілів. У банку, таким чином, не може знаходитися більш восьми автомобілів, включаючи автомобілі двох клієнтів, що обслуговуються в поточний момент касиром. Якщо місце перед банком заповнено до границі, то клієнт, що прибув, вважається втраченим, тому що він відразу ж виїжджає. Початкові умови такі: 1) обидва касири зайняті, тривалість обслуговування для кожного касира нормально розподілена з математичним сподіванням, рівним 1 од. часу, і середньоквадратичним відхиленням, рівним 0,3 од. часу; 2) прибуття першого клієнта заплановано на момент часу 0,1 од. часу; 3) у кожній черзі очікують по два автомобіля.

Визначити такі величини: 1) середнє завантаження кожного касира; 2) середнє число клієнтів у банку; 3) середній інтервал часу між від'їздами клієнтів

від вікон; 4) середній час перебування клієнта в банку; 5) середнє число клієнтів у кожній черзі; 6) відсоток клієнтів, яким відмовлено в обслуговуванні; 7) число змін під'їзних смуг.

3. Для наступного тексту задачі скласти формалізовану модель масового обслуговування та реалізувати її з використанням побудованого універсального алгоритму (40 балів):

У лікарню поступають хворі таких трьох типів: 1) хворі, що пройшли попереднє обстеження і направлені на лікування; 2) хворі, що бажають потрапити в лікарню, але не пройшли повністю попереднє обстеження; 3) хворі, які тільки що поступили на попереднє обстеження. Чисельні характеристики типів хворих наведені в таблиці:

Таблиця 1 – Характеристики типів хворих

Тип хворого	**Відносна частота**	**Середній час реєстрації, хвилин**
1	0,5	15
2	0,1	40
3	0,4	30

При надходженні в приймальне відділення хворий стає в чергу, якщо обидва чергових лікарі зайняті. Лікар, який звільнився, вибирає в першу чергу тих хворих, що вже пройшли попереднє обстеження. Після заповнення різноманітних форм у приймальне відділення хворі 1 типу ідуть прямо в палату, а хворі типів 2 і 3 направляються в лабораторію. Троє супровідних розводять хворих по палатах. Хворим не дозволяється направлятися в палату без супровідного. Якщо всі супровідні зайняті, хворі очікують їхнього звільнення в приймальному відділенні. Як тільки хворий доставлений у палату, він вважається таким, що завершив процес прийому до лікарні.

Хворі, що спрямовуються в лабораторію, не потребують супроводу. Після прибуття в лабораторію хворі стають у чергу в реєстратуру. Після реєстрації вони ідуть у кімнату очікування, де чекають виклику до одного з двох лаборантів. Після здачі аналізів хворі або повертаються в приймальне відділення (якщо їх приймають у лікарню), або залишають лікарню (якщо їм було призначено тільки попереднє обстеження). Після повернення в приймальне відділення хворий, що здав аналізи, розглядається як хворий типу 1. У наступній таблиці приводяться дані по тривалості дій (у хвилинах):

Таблиця 2 – Тривалість дій

Величина	Розподіл
Час між прибуттями в приймальне відділення	Експоненційний з математичним сподіванням 15
Час слідування в палату	Рівномірний від 3 до 8
Час слідування з приймального відділення в лабораторію або з лабораторії в приймальне відділення	Рівномірний від 2 до 5
Час обслуговування в реєстратурі лабораторії	Ерланга з математичним сподіванням 4,5 і k = 3
Час проведення аналізу в лабораторії	Ерланга з математичним сподіванням 4 і k = 2

Визначити час, проведений хворим у системі, тобто інтервал часу, починаючи з надходження і закінчуючи доставкою в палату (для хворих типу 1 і 2) або виходом із лабораторії (для хворих типу 3). Визначити також інтервал між прибуттями хворих у лабораторію.

2. Код програми:

Програму написано мовою Python 3.12 на основі об’єктно-орієнтованого алгоритму імітації з практикуму № 2. Вихідний код і результати розміщено в репозиторії https://github.com/kqjqxb/Computer-simulation (каталог `lab3`). Призначення файлів:

- `fun_rand.py` – генератори випадкових величин (експоненційний, рівномірний, нормальний, Ерланга);
- `item.py` – клас Item: вимога з типом, моментом створення та довільними атрибутами;
- `element.py` – базовий клас Element і клас Route: затримка, маршрути за ймовірністю, пріоритетом, умовою або правилом користувача;
- `elements.py` – Create, Process (багатоканальний, черга з пріоритетом, статистика), Dispose;
- `model.py` – клас Model: принцип найближчої події, обробники подій, період розгону, результати;
- `queueing_theory.py` – формули M/M/c/K, Ерланга B і Кобхема для перевірки алгоритму;
- `runner.py` – серії прогонів і довірчі інтервали;
- `task1_universal.py`, `task2_bank.py`, `task3_hospital.py` – завдання 1–3.

Файл fun_rand.py

```
import math
import random

_rng = random.Random()


def seed(value):
    _rng.seed(value)


def exp(time_mean):
    a = 0.0
    while a == 0.0:
        a = _rng.random()
    return -time_mean * math.log(a)


def unif(time_min, time_max):
    a = 0.0
    while a == 0.0:
        a = _rng.random()
    return time_min + a * (time_max - time_min)


def norm(time_mean, time_deviation):
    return _rng.gauss(time_mean, time_deviation)


def erlang(time_mean, k):
    product = 1.0
    for _ in range(k):
        product *= 1.0 - _rng.random()
    return -time_mean / k * math.log(product)


def uniform01():
    return _rng.random()
```

Файл item.py

```
class Item:
    """Вимога, що рухається моделлю: зберігає тип, момент створення та довільні атрибути."""

    _next_id = 0

    def __init__(self, t_create, kind=1, **attrs):
        self.id = Item._next_id
        Item._next_id += 1
        self.t_create = t_create
        self.kind = kind
        self.t_queue = t_create
        self.attrs = attrs
```

Файл element.py

```
import math

import fun_rand


class Route:
    def __init__(self, target, probability=1.0, priority=0, condition=None):
        self.target = target
        self.probability = probability
        self.priority = priority
        self.condition = condition
        self.count = 0


class Element:
    """Базовий елемент моделі.

    Затримка задається числом (середнє значення закону distribution) або функцією item -> затримка.
    Маршрут виходу обирається за ймовірністю ("probability"), за пріоритетом ("priority": перший за
```

```
пріоритетом блок, що може прийняти вимогу) або функцією routing(item, routes) -> route.
Умова маршруту condition(item) дозволяє розводити вимоги за типом.
"""

_next_id = 0

def __init__(self, name=None, delay=1.0, delay_dev=0.0, distribution="exp", routing="probability"):
    self.id = Element._next_id
    Element._next_id += 1
    self.name = name or f"element{self.id}"
    self.delay = delay
    self.delay_dev = delay_dev
    self.distribution = distribution
    self.routing = routing
    self.routes = []
    self.on_enter = None
    self.tnext = math.inf
    self.tcurr = 0.0
    self.state = 0
    self.quantity = 0

def add_route(self, target, probability=1.0, priority=0, condition=None):
    self.routes.append(Route(target, probability, priority, condition))
    return self

def can_accept(self):
    return True

def choose_route(self, item):
    candidates = [r for r in self.routes if r.condition is None or r.condition(item)]
    if not candidates:
        return None
    if callable(self.routing):
        route = self.routing(item, candidates)
    elif self.routing == "priority":
        ordered = sorted(candidates, key=lambda r: -r.priority)
        route = next((r for r in ordered if r.target.can_accept()), ordered[0])
    else:
        z = fun_rand.uniform01() * sum(r.probability for r in candidates)
        route = candidates[-1]
        cumulative = 0.0
        for r in candidates:
            cumulative += r.probability
            if z < cumulative:
                route = r
                break
    route.count += 1
    return route.target

def send(self, item):
    target = self.choose_route(item)
    if target is not None:
        target.in_act(item)

def get_delay(self, item=None):
    if callable(self.delay):
        return self.delay(item)
    kind = self.distribution.lower()
    if kind == "exp":
        return fun_rand.exp(self.delay)
    if kind == "norm":
        return max(0.0, fun_rand.norm(self.delay, self.delay_dev))
    if kind == "unif":
        return fun_rand.unif(self.delay - self.delay_dev, self.delay + self.delay_dev)
    if kind == "erlang":
        return fun_rand.erlang(self.delay, int(self.delay_dev))
    return self.delay

def in_act(self, item):
    pass

def out_act(self):
    self.quantity += 1

def do_statistics(self, delta):
    pass
```

```
    def reset_statistics(self):
        self.quantity = 0

    def print_info(self):
        print(f"{self.name} state = {self.state} quantity = {self.quantity} tnext = {self.tnext:.4f}")
```

Файл elements.py

```
import math
from collections import defaultdict

from element import Element
from item import Item


class Create(Element):
    def __init__(self, delay, first_time=0.0, item_factory=None, **kwargs):
        super().__init__(delay=delay, **kwargs)
        self.tnext = first_time
        self.item_factory = item_factory or (lambda t: Item(t))

    def out_act(self):
        super().out_act()
        item = self.item_factory(self.tcurr)
        self.tnext = self.tcurr + self.get_delay(item)
        self.send(item)


class Process(Element):
    """Багатоканальний пристрій з чергою обмеженої довжини.

    channels може бути math.inf – тоді елемент моделює затримку без обмеження кількості вимог
    (наприклад, переміщення). queue_key(item) задає пріоритет вибору з черги: першою обслуговується
    вимога з найменшим ключем, за рівних ключів – та, що прийшла раніше.
    """

    def __init__(self, delay, channels=1, maxqueue=math.inf, queue_key=None, record_arrivals=False, **kwargs):
        super().__init__(delay=delay, **kwargs)
        self.channels = channels
        self.maxqueue = maxqueue
        self.queue_key = queue_key
        self.record_arrivals = record_arrivals
        self.in_service = []
        self.queue = []
        self.reset_statistics()

    def reset_statistics(self):
        super().reset_statistics()
        self.failure = 0
        self.arrivals = 0
        self.queue_area = 0.0
        self.busy_area = 0.0
        self.wait = defaultdict(lambda: [0.0, 0])
        self.arrival_times = []

    @property
    def busy(self):
        return len(self.in_service)

    def can_accept(self):
        return self.busy < self.channels or len(self.queue) < self.maxqueue

    def _refresh(self):
        self.tnext = min((t for t, _ in self.in_service), default=math.inf)
        self.state = self.busy

    def _start(self, item):
        record = self.wait[item.kind]
        record[0] += self.tcurr - item.t_queue
        record[1] += 1
        self.in_service.append((self.tcurr + self.get_delay(item), item))

    def accept(self, item):
        """Ставить вимогу на обслуговування або в чергу без перевірки обмеження черги (переведення вимог)."""
        item.t_queue = self.tcurr
        if self.busy < self.channels:
            self._start(item)
```

```
            else:
                self.queue.append(item)
            self._refresh()

        def in_act(self, item):
            if self.on_enter:
                self.on_enter(item)
            self.arrivals += 1
            if self.record_arrivals:
                self.arrival_times.append(self.tcurr)
            if self.can_accept():
                self.accept(item)
            else:
                self.failure += 1

        def _take_from_queue(self):
            if self.queue_key is None:
                return self.queue.pop(0)
            index = min(range(len(self.queue)), key=lambda i: (self.queue_key(self.queue[i]), i))
            return self.queue.pop(index)

        def out_act(self):
            finished = [item for t, item in self.in_service if t == self.tcurr]
            self.in_service = [(t, item) for t, item in self.in_service if t != self.tcurr]
            for _ in finished:
                self.quantity += 1
                if self.queue and self.busy < self.channels:
                    self._start(self._take_from_queue())
            self._refresh()
            for item in finished:
                self.send(item)

        def do_statistics(self, delta):
            self.queue_area += len(self.queue) * delta
            self.busy_area += self.busy * delta

        def mean_queue(self, total_time):
            return self.queue_area / total_time

        def mean_load(self, total_time):
            """Частка часу зайнятості каналів; для необмеженої кількості каналів – середня кількість зайнятих."""
            channels = 1 if self.channels == math.inf else self.channels
            return self.busy_area / (total_time * channels)

        def mean_wait(self, kind=None):
            records = [self.wait[kind]] if kind is not None else list(self.wait.values())
            total, count = sum(r[0] for r in records), sum(r[1] for r in records)
            return total / count if count else 0.0

        def print_info(self):
            print(f"{self.name} state = {self.state} queue = {len(self.queue)} quantity = {self.quantity} "
                  f"failure = {self.failure} tnext = {self.tnext:.4f}")


    class Dispose(Element):
        """Вихід з моделі: фіксує моменти виходу й час перебування вимог за групами group(item)."""

        def __init__(self, name="DISPOSE", group=lambda item: item.kind):
            super().__init__(name=name)
            self.group = group
            self.stats_start = 0.0
            self.reset_statistics()

        def reset_statistics(self):
            super().reset_statistics()
            self.exit_times = []
            self.times = defaultdict(list)

        def in_act(self, item):
            self.quantity += 1
            self.exit_times.append(self.tcurr)
            if item.t_create >= self.stats_start:
                self.times[self.group(item)].append(self.tcurr - item.t_create)

        def all_times(self):
            return [t for values in self.times.values() for t in values]
```

Файл model.py

```
import math
import statistics

from elements import Create, Dispose, Process


class Model:
    """Імітація за принципом найближчої події.

    hooks – функції hook(model), які викликаються після кожної події (правила, що змінюють стан кількох
    елементів одночасно). warmup – тривалість розгону: накопичена до цього моменту статистика скидається.
    """

    def __init__(self, elements, hooks=()):
        self.elements = elements
        self.hooks = list(hooks)
        self.tcurr = 0.0
        self.stats_start = 0.0
        self.in_system_area = 0.0

    def processes(self):
        return [e for e in self.elements if isinstance(e, Process)]

    def in_system(self):
        return sum(len(p.queue) + p.busy for p in self.processes())

    def _accumulate(self, delta):
        for e in self.elements:
            e.do_statistics(delta)
        self.in_system_area += self.in_system() * delta

    def _reset(self, moment):
        self.stats_start = moment
        self.in_system_area = 0.0
        for e in self.elements:
            e.reset_statistics()
            if isinstance(e, Dispose):
                e.stats_start = moment

    def simulate(self, time, warmup=0.0, trace_events=0):
        events = 0
        while self.tcurr < time:
            tnext = min(e.tnext for e in self.elements)
            t_stop = min(tnext, time)
            if self.tcurr < warmup <= t_stop:
                self._accumulate(warmup - self.tcurr)
                self._reset(warmup)
                self._accumulate(t_stop - warmup)
            else:
                self._accumulate(t_stop - self.tcurr)
            if tnext > time:
                self.tcurr = time
                break

            self.tcurr = tnext
            for e in self.elements:
                e.tcurr = self.tcurr
            active = [e for e in self.elements if e.tnext == self.tcurr]
            for e in active:
                e.out_act()
            for hook in self.hooks:
                hook(self)

            events += 1
            if events <= trace_events:
                names = ", ".join(e.name for e in active)
                print(f"\nIt's time for event in {names}, time = {self.tcurr:.4f}")
                for e in self.elements:
                    e.print_info()
        for e in self.elements:
            e.tcurr = self.tcurr

    def results(self):
        total = self.tcurr - self.stats_start
        disposes = [e for e in self.elements if isinstance(e, Dispose)]
```

```
        times = [t for d in disposes for t in d.all_times()]
        exits = sorted(t for d in disposes for t in d.exit_times)
        out = {
            "time": total,
            "created": sum(e.quantity for e in self.elements if isinstance(e, Create)),
            "disposed": sum(d.quantity for d in disposes),
            "failures": sum(p.failure for p in self.processes()),
            "mean_in_system": self.in_system_area / total,
            "mean_time_in_system": statistics.mean(times) if times else math.nan,
            "mean_exit_interval": (exits[-1] - exits[0]) / (len(exits) - 1) if len(exits) > 1 else math.nan,
            "processes": {},
            "groups": {},
        }
        for p in self.processes():
            out["processes"][p.name] = {
                "arrivals": p.arrivals, "served": p.quantity, "failure": p.failure,
                "mean_queue": p.mean_queue(total), "mean_load": p.mean_load(total),
                "mean_wait": p.mean_wait(),
                "wait_by_kind": {k: v[0] / v[1] for k, v in p.wait.items() if v[1]},
            }
        for d in disposes:
            for group, values in d.times.items():
                out["groups"][f"{d.name}: {group}"] = statistics.mean(values)
        return out
```

Файл queueing_theory.py

```
"""Аналітичні формули теорії масового обслуговування для перевірки універсального алгоритму."""
import math


def mmck(arrival_rate, service_mean, channels=1, capacity_queue=math.inf):
    """Система M/M/c/K: c каналів, черга до capacity_queue місць."""
    a = arrival_rate * service_mean
    c = channels
    if capacity_queue == math.inf:
        rho = a / c
        p0 = 1 / (sum(a ** n / math.factorial(n) for n in range(c))
                  + a ** c / (math.factorial(c) * (1 - rho)))
        lq = p0 * a ** c * rho / (math.factorial(c) * (1 - rho) ** 2)
        return {"failure_probability": 0.0, "mean_queue": lq, "mean_load": rho}
    k = c + int(capacity_queue)
    weights = [a ** n / math.factorial(n) if n <= c else a ** n / (math.factorial(c) * c ** (n - c))
               for n in range(k + 1)]
    p = [w / sum(weights) for w in weights]
    lq = sum((n - c) * p[n] for n in range(c + 1, k + 1))
    busy = sum(min(n, c) * p[n] for n in range(k + 1))
    return {"failure_probability": p[k], "mean_queue": lq, "mean_load": busy / c,
            "mean_in_system": lq + busy}


def erlang_b(channels, load):
    """Ймовірність втрати в системі з відмовами M/M/c/0 (формула Ерланга B, рекурентно)."""
    b = 1.0
    for n in range(1, channels + 1):
        b = load * b / (n + load * b)
    return b


def priority_waits(rates, service_means, second_moments):
    """Середній час очікування класів у M/G/1 з відносними пріоритетами (формула Кобхема).

    Клас 0 має найвищий пріоритет.
    """
    w0 = sum(lam * m2 for lam, m2 in zip(rates, second_moments)) / 2
    waits, sigma = [], 0.0
    for lam, mean in zip(rates, service_means):
        previous = sigma
        sigma += lam * mean
        waits.append(w0 / ((1 - previous) * (1 - sigma)))
    return waits
```

Файл runner.py

```
import statistics
```

```
import fun_rand

# квантилі t-розподілу Стьюдента рівня 0.975 за кількістю ступенів вільності
T_95 = {1: 12.706, 2: 4.303, 3: 3.182, 4: 2.776, 5: 2.571, 6: 2.447, 7: 2.365, 8: 2.306, 9: 2.262, 19: 2.093}


def replicate(build_model, time, seeds, warmup=0.0, collect=None):
    """Запускає модель з кількома зернами генератора; повертає collect(model) для кожного прогону
    (за замовчуванням – model.results())."""
    runs = []
    for s in seeds:
        fun_rand.seed(s)
        model = build_model()
        model.simulate(time, warmup=warmup)
        runs.append(collect(model) if collect else model.results())
    return runs


def summary(values):
    """Середнє та півширина 95 % довірчого інтервалу за t-розподілом Стьюдента."""
    mean = statistics.mean(values)
    if len(values) < 2:
        return mean, 0.0
    return mean, T_95.get(len(values) - 1, 1.96) * statistics.stdev(values) / len(values) ** 0.5


def metric(runs, path):
    values = []
    for r in runs:
        value = r
        for key in path:
            value = value[key]
        values.append(value)
    return summary(values)
```

Файл task1_universal.py

```
"""Завдання 1: перевірка універсального алгоритму імітації на моделях з відомим аналітичним розв'язком."""
import json
from pathlib import Path

import fun_rand
from elements import Create, Dispose, Process
from item import Item
from model import Model
from queueing_theory import erlang_b, mmck, priority_waits
from runner import metric, replicate

RESULTS = Path(__file__).resolve().parent / "results"
TIME, WARMUP, SEEDS = 50_000.0, 1_000.0, range(1, 21)


def run(build):
    return replicate(build, TIME, SEEDS, warmup=WARMUP)


def multichannel():
    """Багатоканальний пристрій з обмеженою чергою: M/M/3/3."""
    def build():
        create = Create(1.0, name="CREATE")
        process = Process(2.5, channels=3, maxqueue=3, name="P")
        dispose = Dispose()
        create.add_route(process)
        process.add_route(dispose)
        return Model([create, process, dispose])

    runs = run(build)
    th = mmck(1.0, 2.5, 3, 3)
    failure = [r["processes"]["P"]["failure"] / r["processes"]["P"]["arrivals"] for r in runs]
    rows = [("ймовірність відмови", sum(failure) / len(failure), None, th["failure_probability"])]
    for label, key in [("завантаження каналу", "mean_load"), ("середня черга", "mean_queue")]:
        rows.append((label, *metric(runs, ["processes", "P", key]), th[key]))
    return "Багатоканальний пристрій: M/M/3/3, λ = 1, τ = 2,5", rows


def priority_routing():
    """Вибір маршруту за пріоритетом: основний пристрій, а якщо він зайнятий – резервний
```

```
    (система з відмовами)."""
    def build():
        create = Create(1.0, name="CREATE", routing="priority")
        main = Process(1.0, maxqueue=0, name="MAIN")
        reserve = Process(1.0, maxqueue=0, name="RESERVE")
        dispose = Dispose()
        create.add_route(main, priority=2).add_route(reserve, priority=1)
        main.add_route(dispose)
        reserve.add_route(dispose)
        return Model([create, main, reserve, dispose])

    runs = run(build)
    b1, b2 = erlang_b(1, 1.0), erlang_b(2, 1.0)
    lost = [r["failures"] / r["created"] for r in runs]
    rows = [("завантаження основного", *metric(runs, ["processes", "MAIN", "mean_load"]), 1.0 * (1 - b1)),
            ("завантаження резервного", *metric(runs, ["processes", "RESERVE", "mean_load"]),
             1.0 * (b1 - b2)),
            ("ймовірність втрати", sum(lost) / len(lost), None, b2)]
    return "Маршрут за пріоритетом: два пристрої без черги, λ = 1, τ = 1 (формула Ерланга B)", rows


def probability_routing():
    """Вибір маршруту за ймовірністю: розщеплення пуассонівського потоку на три системи M/M/1."""
    shares = (0.2, 0.3, 0.5)

    def build():
        create = Create(1.0, name="CREATE")
        dispose = Dispose()
        nodes = [Process(1.0, name=f"P{i + 1}") for i in range(3)]
        for node, share in zip(nodes, shares):
            create.add_route(node, probability=share)
            node.add_route(dispose)
        return Model([create, *nodes, dispose])

    runs = run(build)
    rows = []
    for i, share in enumerate(shares):
        th = mmck(share, 1.0)
        arrivals = [r["processes"][f"P{i + 1}"]["arrivals"] / r["created"] for r in runs]
        rows.append((f"P{i + 1}: частка вимог", sum(arrivals) / len(arrivals), None, share))
        rows.append((f"P{i + 1}: середня черга", *metric(runs, ["processes", f"P{i + 1}", "mean_queue"]),
                     th["mean_queue"]))
    return "Маршрут за ймовірністю 0,2 / 0,3 / 0,5, λ = 1, τ = 1", rows


def priority_queue():
    """Пріоритетний вибір з черги: M/M/1 з двома класами вимог і відносним пріоритетом класу 1."""
    rate, share1 = 0.8, 0.3

    def build():
        create = Create(1 / rate, name="CREATE",
                        item_factory=lambda t: Item(t, 1 if fun_rand.uniform01() < share1 else 2))
        process = Process(1.0, queue_key=lambda item: item.kind, name="P")
        dispose = Dispose()
        create.add_route(process)
        process.add_route(dispose)
        return Model([create, process, dispose])

    runs = run(build)
    th = priority_waits([rate * share1, rate * (1 - share1)], [1.0, 1.0], [2.0, 2.0])
    rows = [(f"очікування класу {k}", *metric(runs, ["processes", "P", "wait_by_kind", k]), th[k - 1])
            for k in (1, 2)]
    return "Пріоритетна черга: два класи, λ = 0,8, τ = 1 (формула Кобхема)", rows


def main():
    fun_rand.seed(1)
    out = []
    print(f"ПЕРЕВІРКА УНІВЕРСАЛЬНОГО АЛГОРИТМУ: {len(SEEDS)} прогонів по {TIME:.0f} одиниць часу, "
          f"розгін {WARMUP:.0f}")
    for check in (multichannel, priority_routing, probability_routing, priority_queue):
        title, rows = check()
        print(f"\n{title}")
        for label, sim, ci, theory in rows:
            interval = f" ± {ci:.4f}" if ci is not None else ""
            print(f"  {label:26} імітація = {sim:.4f}{interval}, теорія = {theory:.4f}")
        out.append({"title": title,
```

```
                    "rows": [{"metric": m, "sim": s, "ci": c, "theory": t} for m, s, c, t in rows]})
    with open(RESULTS / "task1.json", "w", encoding="utf-8") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

Файл task2_bank.py

```
"""Завдання 2: банк для автомобілістів з двома віконцями та під'їзними смугами."""
import json
import statistics
from pathlib import Path

import fun_rand
from elements import Create, Dispose, Process
from item import Item
from model import Model
from queueing_theory import mmck
from runner import replicate, summary

RESULTS = Path(__file__).resolve().parent / "results"
ARRIVAL_MEAN, SERVICE_MEAN = 0.5, 0.3
LANE_QUEUE = 3  # місць очікування на смузі, крім автомобіля біля вікна
TIME, SEEDS = 1000.0, range(1, 21)


def cars(lane):
    return len(lane.queue) + lane.busy


def choose_lane(item, routes):
    """Коротша смуга; за рівної кількості автомобілів – перша."""
    first, second = routes
    return second if cars(second.target) < cars(first.target) else first


class LaneSwitch:
    """Останній автомобіль переїжджає на іншу смугу, якщо на його смузі щонайменше
    на два автомобілі більше."""

    def __init__(self, lanes):
        self.lanes = lanes
        self.count = 0

    def __call__(self, model):
        a, b = self.lanes
        for source, target in ((a, b), (b, a)):
            while source.queue and cars(source) - cars(target) >= 2:
                target.accept(source.queue.pop())
                self.count += 1


def build():
    cashiers = [Process(SERVICE_MEAN, maxqueue=LANE_QUEUE, name=f"Касир {i}") for i in (1, 2)]
    create = Create(ARRIVAL_MEAN, first_time=0.1, name="Прибуття", routing=choose_lane)
    bank_exit = Dispose("Від'їзд", group=lambda item: "клієнти")
    for cashier in cashiers:
        create.add_route(cashier)
        cashier.add_route(bank_exit)
        # початкові умови: касир зайнятий, обслуговування ~ N(1; 0.3), у черзі два автомобілі
        cashier.in_service.append((max(0.0, fun_rand.norm(1.0, 0.3)), Item(0.0)))
        cashier.queue.extend(Item(0.0) for _ in range(2))
        cashier.tnext = cashier.in_service[0][0]
        cashier.state = 1
    switch = LaneSwitch(cashiers)
    model = Model([create, *cashiers, bank_exit], hooks=[switch])
    model.lane_switch = switch
    return model


def collect(model):
    r = model.results()
    cashiers = model.elements[1:3]
    arrivals = r["created"]
    return {
```

```
        "load": [r["processes"][c.name]["mean_load"] for c in cashiers],
        "mean_in_bank": r["mean_in_system"],
        "exit_interval": r["mean_exit_interval"],
        "time_in_bank": r["mean_time_in_system"],
        "queues": [r["processes"][c.name]["mean_queue"] for c in cashiers],
        "refused_percent": 100 * r["failures"] / arrivals,
        "lane_switches": model.lane_switch.count,
        "created": arrivals, "served": r["disposed"],
    }


def main():
    fun_rand.seed(2)
    model = build()
    print(f"Банк: прибуття exp({ARRIVAL_MEAN}), обслуговування exp({SERVICE_MEAN}), на смузі до {LANE_QUEUE} "
          f"автомобілів у черзі, час моделювання {TIME:.0f}")
    model.simulate(TIME, trace_events=4)
    one = collect(model)

    runs = replicate(build, TIME, SEEDS, collect=collect)

    def stat(key, index=None):
        return summary([r[key] if index is None else r[key][index] for r in runs])

    table = [
        ("1) завантаження касира 1", stat("load", 0)),
        ("1) завантаження касира 2", stat("load", 1)),
        ("2) середнє число клієнтів у банку", stat("mean_in_bank")),
        ("3) середній інтервал між від'їздами", stat("exit_interval")),
        ("4) середній час перебування в банку", stat("time_in_bank")),
        ("5) середня черга на смузі 1", stat("queues", 0)),
        ("5) середня черга на смузі 2", stat("queues", 1)),
        ("6) відсоток клієнтів, яким відмовлено", stat("refused_percent")),
        ("7) число змін під'їзних смуг", stat("lane_switches")),
    ]
    print(f"\nОдин прогін: {json.dumps(one, ensure_ascii=False)}")
    print(f"\nРЕЗУЛЬТАТИ ({len(SEEDS)} прогонів по {TIME:.0f} од. часу, 95 % довірчий інтервал):")
    for label, (mean, ci) in table:
        print(f"  {label:40} {mean:9.4f} ± {ci:.4f}")

    rate = statistics.mean(r["created"] / TIME for r in runs)
    served_rate = statistics.mean(r["served"] / TIME for r in runs)
    little = served_rate * table[4][1][0]
    print(f"\nПеревірка за формулою Літтла: λ_обсл · W = {served_rate:.4f} · {table[4][1][0]:.4f} = "
          f"{little:.4f}, "
          f"середнє число в банку = {table[2][1][0]:.4f}")
    print(f"Інтенсивність прибуття {rate:.4f} (задано {1 / ARRIVAL_MEAN}); 1 / інтервал між від'їздами = "
          f"{1 / table[3][1][0]:.4f}, інтенсивність обслужених = {served_rate:.4f}")
    approx = mmck(1 / ARRIVAL_MEAN, SERVICE_MEAN, 2, 2 * LANE_QUEUE)
    print(f"Наближення M/M/2/8 зі спільною чергою: завантаження {approx['mean_load']:.4f}, "
          f"відмови {100 * approx['failure_probability']:.4f} %, середнє число в системі "
          f"{approx['mean_in_system']:.4f}")

    with open(RESULTS / "task2.json", "w", encoding="utf-8") as f:
        json.dump({"single": one, "table": [{"metric": m, "mean": v[0], "ci": v[1]} for m, v in table],
                   "little": {"served_rate": served_rate, "time": table[4][1][0], "product": little,
                              "mean_in_bank": table[2][1][0]},
                   "arrival_rate": rate, "approx": approx}, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

Файл task3_hospital.py

```
"""Завдання 3: приймання хворих у лікарню (приймальне відділення, супровідні, лабораторія)."""
import json
import math
import statistics
from pathlib import Path

import fun_rand
from elements import Create, Dispose, Process
from item import Item
from model import Model
from runner import replicate, summary
```

```
RESULTS = Path(__file__).resolve().parent / "results"
ARRIVAL_MEAN = 15.0
TYPES = {1: (0.5, 15.0), 2: (0.1, 40.0), 3: (0.4, 30.0)}  # тип: (відносна частота, середній час реєстрації)
TIME, WARMUP, SEEDS = 100_000.0, 2_000.0, range(1, 21)

# очікуване завантаження з балансу потоків: частка хворих, що проходять пристрій, / 15 · τ / кількість каналів
LAB_SHARE, WARD_SHARE = TYPES[2][0] + TYPES[3][0], TYPES[1][0] + TYPES[2][0]
EXPECTED_LOAD = {
    "Приймальне відділення": (sum(share * mean for share, mean in TYPES.values())
                              + TYPES[2][0] * TYPES[1][1]) / ARRIVAL_MEAN / 2,
    "Супровідні": WARD_SHARE / ARRIVAL_MEAN * 5.5 / 3,
    "Перехід до лабораторії": LAB_SHARE / ARRIVAL_MEAN * 3.5,
    "Реєстратура лабораторії": LAB_SHARE / ARRIVAL_MEAN * 4.5,
    "Лаборанти": LAB_SHARE / ARRIVAL_MEAN * 4.0 / 2,
    "Повернення до приймального": TYPES[2][0] / ARRIVAL_MEAN * 3.5,
}


def new_patient(t):
    z, cumulative = fun_rand.uniform01(), 0.0
    for kind, (share, _) in TYPES.items():
        cumulative += share
        if z < cumulative:
            return Item(t, kind, origin=kind)
    return Item(t, 3, origin=3)


def become_type1(item):
    item.kind = 1


def build():
    create = Create(ARRIVAL_MEAN, name="Надходження", item_factory=new_patient)
    reception = Process(lambda item: fun_rand.exp(TYPES[item.kind][1]), channels=2,
                        queue_key=lambda item: 0 if item.kind == 1 else 1, name="Приймальне відділення")
    escort = Process(lambda item: fun_rand.unif(3, 8), channels=3, name="Супровідні")
    to_lab = Process(lambda item: fun_rand.unif(2, 5), channels=math.inf, name="Перехід до лабораторії")
    registry = Process(lambda item: fun_rand.erlang(4.5, 3), record_arrivals=True,
                       name="Реєстратура лабораторії")
    lab = Process(lambda item: fun_rand.erlang(4.0, 2), channels=2, name="Лаборанти")
    back = Process(lambda item: fun_rand.unif(2, 5), channels=math.inf, name="Повернення до приймального")
    back.on_enter = become_type1
    ward = Dispose("Палата", group=lambda item: item.attrs["origin"])
    lab_exit = Dispose("Вихід з лабораторії", group=lambda item: item.attrs["origin"])

    create.add_route(reception)
    reception.add_route(escort, condition=lambda item: item.kind == 1)
    reception.add_route(to_lab, condition=lambda item: item.kind != 1)
    escort.add_route(ward)
    to_lab.add_route(registry)
    registry.add_route(lab)
    lab.add_route(back, condition=lambda item: item.kind == 2)
    lab.add_route(lab_exit, condition=lambda item: item.kind == 3)
    back.add_route(reception)
    return Model([create, reception, escort, to_lab, registry, lab, back, ward, lab_exit])


def collect(model):
    r = model.results()
    elements = {e.name: e for e in model.elements}
    ward, lab_exit = elements["Палата"], elements["Вихід з лабораторії"]
    lab_arrivals = elements["Реєстратура лабораторії"].arrival_times
    out = {
        "type1": statistics.mean(ward.times[1]),
        "type2": statistics.mean(ward.times[2]),
        "type3": statistics.mean(lab_exit.times[3]),
        "all": r["mean_time_in_system"],
        "lab_interval": (lab_arrivals[-1] - lab_arrivals[0]) / (len(lab_arrivals) - 1),
        "patients": r["created"],
    }
    for name, p in r["processes"].items():
        out[f"load: {name}"] = p["mean_load"]
        out[f"queue: {name}"] = p["mean_queue"]
        out[f"wait: {name}"] = p["mean_wait"]
    return out
```

```
def main():
    fun_rand.seed(3)
    model = build()
    print(f"Лікарня: надходження exp({ARRIVAL_MEAN}) хв, типи хворих {TYPES}")
    model.simulate(300.0, trace_events=4)

    runs = replicate(build, TIME, SEEDS, warmup=WARMUP, collect=collect)

    def stat(key):
        return summary([r[key] for r in runs])

    print(f"\nРЕЗУЛЬТАТИ ({len(SEEDS)} прогонів по {TIME:.0f} хв, розгін {WARMUP:.0f} хв, "
          "95 % довірчий інтервал):")
    main_rows = [
        ("час у системі, хворі типу 1", stat("type1")),
        ("час у системі, хворі типу 2", stat("type2")),
        ("час у системі, хворі типу 3", stat("type3")),
        ("час у системі, усі хворі", stat("all")),
        ("інтервал між прибуттями в лабораторію", stat("lab_interval")),
    ]
    for label, (mean, ci) in main_rows:
        print(f"  {label:40} {mean:9.3f} ± {ci:.3f} хв")
    detail = []
    print("\nХарактеристики пристроїв:")
    for name in EXPECTED_LOAD:
        values = {k: stat(f"{k}: {name}") for k in ("load", "queue", "wait")}
        detail.append({"element": name, "expected_load": EXPECTED_LOAD[name],
                       **{k: v[0] for k, v in values.items()},
                       **{f"{k}_ci": v[1] for k, v in values.items()}})
        print(f"  {name:28} завантаження {values['load'][0]:.4f} ± {values['load'][1]:.4f} "
              f"(баланс потоків {EXPECTED_LOAD[name]:.4f}), черга {values['queue'][0]:.4f}, "
              f"очікування {values['wait'][0]:.3f} хв")

    print(f"\nОчікуваний інтервал між прибуттями в лабораторію = {ARRIVAL_MEAN} / ({LAB_SHARE:.1f}) = "
          f"{ARRIVAL_MEAN / LAB_SHARE:.1f} хв")

    with open(RESULTS / "task3.json", "w", encoding="utf-8") as f:
        json.dump({"main": [{"metric": m, "mean": v[0], "ci": v[1]} for m, v in main_rows], "detail": detail,
                   "expected_lab_interval": ARRIVAL_MEAN / LAB_SHARE}, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

3. Результати роботи програми:

Завдання 1:

Універсальний алгоритм розширює модель з практикуму № 2 у трьох напрямах.

- Вимоги стали об’єктами класу Item. Кожна вимога зберігає тип, момент створення та довільні атрибути, тож модель може обчислювати час перебування окремих вимог, розводити вимоги за типом і змінювати тип вимоги під час руху моделлю. Затримку елемента можна задати не лише середнім значенням закону, а й функцією від вимоги (наприклад, час реєстрації залежить від типу хворого).
- Process обслуговує вимоги кількома каналами: список `in_service` містить пари «момент закінчення обслуговування, вимога», а кількість каналів може бути нескінченною (`math.inf`) – так моделюються переміщення, у яких вимоги не заважають одна одній. Черга зберігає самі

вимоги; функція `queue_key` задає пріоритет вибору з черги (за рівних пріоритетів – порядок надходження). Метод `accept` дозволяє перевести вимогу в інший пристрій поза звичайним маршрутом (перешикування між чергами).

- Маршрут виходу описується об’єктами Route з ймовірністю, пріоритетом та умовою за типом вимоги. Режим `routing="probability"` обирає маршрут за ймовірністю, `routing="priority"` – перший за пріоритетом блок, який може прийняти вимогу (має вільний канал або місце в черзі), а функція `routing(item, routes)` задає довільне правило (наприклад, вибір коротшої черги). Клас Model додатково підтримує обробники `hooks`, що виконуються після кожної події, та період розгону `warmup`, після якого статистика збирається заново.

Кожну складову алгоритму перевірено на моделі, для якої відомий точний аналітичний розв’язок (20 прогонів по 50000 одиниць часу з розгоном 1000; для середніх наведено 95 % довірчий інтервал):

Таблиця 3 – Перевірка алгоритму за аналітичними розв’язками

Показник	**Імітація**	**Теорія**
Багатоканальний пристрій: M/M/3/3, λ = 1, τ = 2,5		
ймовірність відмови	0,1028	0,1024
завантаження каналу	0,7502 ± 0,0012	0,7480
середня черга	0,7040 ± 0,0045	0,7005
Маршрут за пріоритетом: два пристрої без черги, λ = 1, τ = 1 (формула Ерланга B)		
завантаження основного	0,5003 ± 0,0007	0,5000
завантаження резервного	0,3013 ± 0,0008	0,3000
ймовірність втрати	0,2006	0,2000
Маршрут за ймовірністю 0,2 / 0,3 / 0,5, λ = 1, τ = 1		
P1: частка вимог	0,2000	0,2000
P1: середня черга	0,0491 ± 0,0007	0,0500
P2: частка вимог	0,2999	0,3000
P2: середня черга	0,1288 ± 0,0023	0,1286
P3: частка вимог	0,5001	0,5000
P3: середня черга	0,4978 ± 0,0076	0,5000
Пріоритетна черга: два класи, λ = 0,8, τ = 1 (формула Кобхема)		
очікування класу 1	1,0561 ± 0,0114	1,0526
очікування класу 2	5,2374 ± 0,1654	5,2632

Усі результати імітації збігаються з теоретичними значеннями (табл. 3). Окремо перевірено завантаження каналу в M/M/3/3, яке вийшло на межі довірчого

інтервалу: повторення з 40 іншими зернами по 100000 одиниць часу дало 0,7476 ± 0,0007 при теоретичному 0,7480. Вибір за пріоритетом відтворює систему з відмовами Ерланга (втрачаються лише вимоги, що застали обидва пристрої зайнятими), вибір за ймовірністю – розщеплення пуассонівського потоку, а пріоритетна черга – формулу Кобхема для відносних пріоритетів: вимоги класу 1 чекають у п’ять разів менше за вимоги класу 2.

Завдання 2:

Формалізована модель масового обслуговування банку (рисунок 1, таблиця 4):

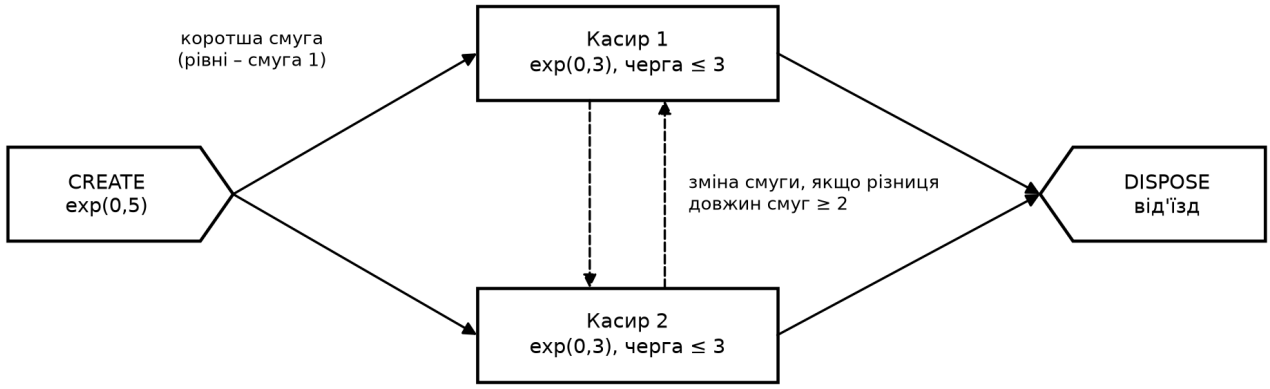


Рисунок 1 – Формалізована модель банку для автомобілістів

Таблиця 4 – Елементи формалізованої моделі банку

Елемент	**Параметри**	**Правила**
Прибуття (Create)	інтервал exp(0,5); перше прибуття в момент 0,1	смуга з меншою кількістю автомобілів; за рівної кількості – смуга 1
Касир 1, Касир 2 (Process)	1 канал, обслуговування exp(0,3), до 3 автомобілів у черзі	якщо обидві смуги заповнені, клієнт втрачається
Зміна смуги (обробник події)	після кожної події	останній автомобіль переїжджає, якщо на його смузі щонайменше на 2 автомобілі більше
Від’їзд (Dispose)	–	фіксує моменти від’їзду та час перебування в банку

Довжина смуги – це кількість автомобілів на ній, включно з автомобілем біля вікна. Якщо порівнювати лише черги, то клієнт, що застав касира 1 зайнятим, а касира 2 вільним (обидві черги порожні), мав би стати за першим автомобілем на смузі 1 і чекати при вільному касирі 2, що суперечить змісту задачі. Обмеження «до трьох автомобілів у черзі на смузі» разом з автомобілем біля вікна дає саме

вісім автомобілів у банку. Початкові умови задаються безпосередньо в стані елементів: у кожного касира автомобіль з часом обслуговування N(1; 0,3) і два автомобілі в черзі; ці шість клієнтів вважаються такими, що прибули в момент 0.

Вибір смуги реалізовано функцією `choose_lane`, переданою як правило маршрутизації, а зміну смуги – обробником `LaneSwitch`, який після кожної події переводить останній автомобіль методом `accept`. Початок трасування:

```
Банк: прибуття exp(0.5), обслуговування exp(0.3), на смузі до 3 автомобілів у черзі, час
моделювання 1000

It's time for event in Прибуття, time = 0.1000
Прибуття state = 0 quantity = 1 tnext = 1.5363
Касир 1 state = 1 queue = 3 quantity = 0 failure = 0 tnext = 1.7015
Касир 2 state = 1 queue = 2 quantity = 0 failure = 0 tnext = 0.8011
Від'їзд state = 0 quantity = 0 tnext = inf

It's time for event in Касир 2, time = 0.8011
Прибуття state = 0 quantity = 1 tnext = 1.5363
Касир 1 state = 1 queue = 2 quantity = 0 failure = 0 tnext = 1.7015
Касир 2 state = 1 queue = 2 quantity = 1 failure = 0 tnext = 1.5411
Від'їзд state = 0 quantity = 1 tnext = inf
```

У першій події новий клієнт обрав смугу 1 (обидві смуги мали по три автомобілі). Після від’їзду клієнта від касира 2 різниця смуг стала $4 - 2 = 2$, і останній автомобіль переїхав на смугу 2.

Тривалість години пік у задачі не задана, тому моделювався інтервал 1000 од. часу (близько 2000 клієнтів) – 20 прогонів з різними зернами генератора:

Таблиця 5 – Результати моделювання банку (95 % довірчий інтервал)

Величина	**Значення**
1) завантаження касира 1	0,4052 ± 0,0045
1) завантаження касира 2	0,2023 ± 0,0052
2) середнє число клієнтів у банку	0,6757 ± 0,0122
3) середній інтервал між від'їздами	0,4956 ± 0,0044
4) середній час перебування в банку	0,3351 ± 0,0052
5) середня черга на смузі 1	0,0468 ± 0,0032
5) середня черга на смузі 2	0,0214 ± 0,0025
6) відсоток клієнтів, яким відмовлено	0,0372 ± 0,0292
7) число змін під'їзних смуг	107,6000 ± 6,1405

Касир 1 завантажений удвічі більше за касира 2 (табл. 5): за рівної кількості автомобілів, зокрема коли обидві смуги порожні, клієнти обирають смугу 1, а другий касир працює лише тоді, коли перший зайнятий. Черги короткі, відмови трапляються рідко (0,04 %), а переїзди між смугами відбуваються часто –

здебільшого коли касир 2 звільняється, а на смузі 1 один автомобіль обслуговується і один чекає.

Для верифікації перевірено узгодженість незалежно обчислених величин. За формулою Літтла середнє число клієнтів у банку має дорівнювати добутку інтенсивності обслужених клієнтів на середній час перебування: 2,0158 · 0,3351 = 0,6754, а виміряне значення становить 0,6757. Середній інтервал між від'їздами (0,4956) обернений до інтенсивності обслужених клієнтів (≈ 2,02 клієнта за одиницю часу при заданій інтенсивності прибуття 2). Сумарне завантаження касирів 0,405 + 0,202 = 0,607 відповідає $\lambda \cdot \tau = 2 \cdot 0{,}3 = 0{,}6$; невелике перевищення дають шість початкових клієнтів (обслуговування з середнім 1 од. часу) і фактична інтенсивність прибуття в цих прогонах 2,011. Для порівняння, система M/M/2/8 зі спільною чергою дає 0,6591 клієнта в системі, що близько до отриманих 0,676.

Завдання 3:

Формалізована модель масового обслуговування лікарні (рисунок 2, таблиця 7):

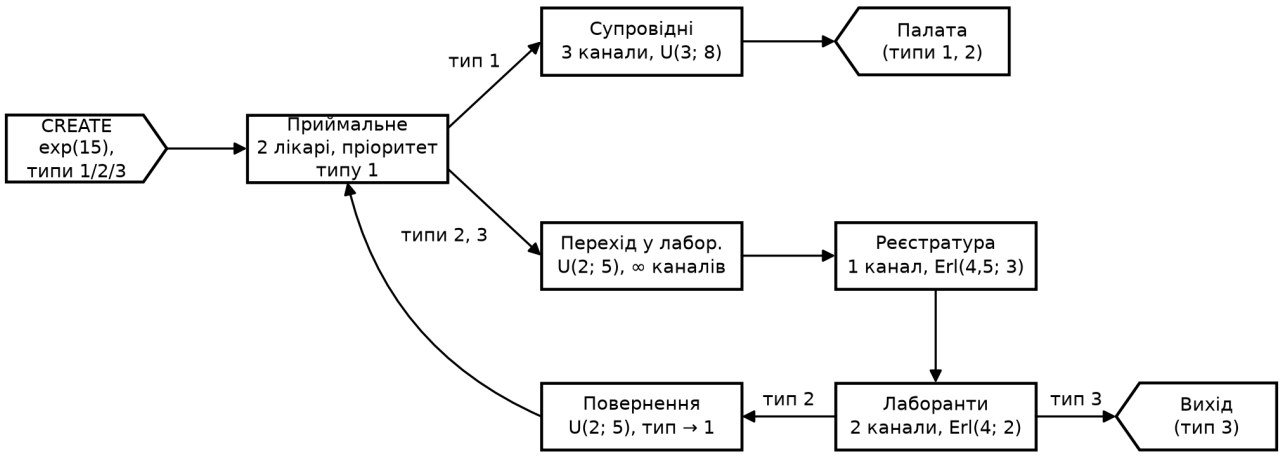


Рисунок 2 – Формалізована модель приймання хворих у лікарню

Таблиця 6 – Елементи формалізованої моделі лікарні

Елемент	Канали	Затримка, хв	Правила
Надходження (Create)	–	exp(15)	тип 1, 2, 3 з ймовірностями 0,5 / 0,1 / 0,4
Приймальне відділення	2 лікарі	exp(15), exp(40), exp(30) за типом	з черги першими – хворі типу 1; тип 1 – до супровідних, типи 2, 3 – до лабораторії
Супровідні	3	U(3; 8)	черга в приймальному відділенні; далі – палата
Перехід до лабораторії	∞	U(2; 5)	–
Реєстратура лабораторії	1	Erlang(4,5; k = 3)	фіксуються моменти прибуття в лабораторію
Лаборанти	2	Erlang(4; k = 2)	тип 2 – повернення, тип 3 – вихід з лікарні
Повернення до приймального	∞	U(2; 5)	хворий стає хворим типу 1
Палата, Вихід з лабораторії (Dispose)	–	–	час перебування за початковим типом хворого

Закон розподілу часу реєстрації в приймальному відділенні в задачі не вказано, задано лише середні значення, тому прийнято експоненційний закон. Супровідний зайнятий на час слідування хворого в палату (час повернення супровідного не задано). Хворий типу 2 після лабораторії отримує тип 1 (у момент входу в елемент повернення), тому в приймальному відділенні він має пріоритет і проходить реєстрацію із середнім часом 15 хв, а для статистики зберігається його початковий тип. Тип хворого в моделі – атрибут вимоги, за яким працюють умовні маршрути, пріоритетна черга та затримка реєстрації.

Початок трасування:

```
Лікарня: надходження exp(15.0) хв, типи хворих {1: (0.5, 15.0), 2: (0.1, 40.0), 3: (0.4, 30.0)}

It's time for event in Надходження, time = 0.0000
Надходження state = 0 quantity = 1 tnext = 9.1258
Приймальне відділення state = 1 queue = 0 quantity = 0 failure = 0 tnext = 7.5647
Супровідні state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
Перехід до лабораторії state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
Реєстратура лабораторії state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
Лаборанти state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
Повернення до приймального state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
Палата state = 0 quantity = 0 tnext = inf
Вихід з лабораторії state = 0 quantity = 0 tnext = inf

It's time for event in Приймальне відділення, time = 7.5647
Надходження state = 0 quantity = 1 tnext = 9.1258
Приймальне відділення state = 0 queue = 0 quantity = 1 failure = 0 tnext = inf
Супровідні state = 1 queue = 0 quantity = 0 failure = 0 tnext = 10.8923
Перехід до лабораторії state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
Реєстратура лабораторії state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
Лаборанти state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
Повернення до приймального state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
Палата state = 0 quantity = 0 tnext = inf
Вихід з лабораторії state = 0 quantity = 0 tnext = inf
```

Моделювання виконано 20 разів по 100000 хв (≈ 6700 хворих за прогін); перші 2000 хв – період розгону, щоб результат не залежав від порожнього

початкового стану лікарні. Час перебування рахується для хворих, які надійшли після розгону.

Таблиця 7 – Результати моделювання лікарні, хв (95 % ДІ)

Величина	Значення
час у системі, хворі типу 1	35,38 ± 0,61
час у системі, хворі типу 2	178,43 ± 8,27
час у системі, хворі типу 3	130,16 ± 6,71
час у системі, усі хворі	87,63 ± 3,85
інтервал між прибуттями в лабораторію	29,94 ± 0,23

Таблиця 8 – Характеристики пристроїв лікарні

Пристрій	Завантаження (імітація)	Завантаження (баланс потоків)	Середня черга	Середнє очікування, хв
Приймальне відділення	0,8388 ± 0,0071	0,8333	3,512	47,91
Супровідні	0,0731 ± 0,0006	0,0733	0,000	0,00
Перехід до лабораторії	0,1168 ± 0,0010	0,1167	0,000	0,00
Реєстратура лабораторії	0,1506 ± 0,0013	0,1500	0,017	0,52
Лаборанти	0,0666 ± 0,0006	0,0667	0,000	0,01
Повернення до приймального	0,0231 ± 0,0005	0,0233	0,000	0,00

Хворі типу 1 перебувають у системі в середньому 35,4 хв, типу 2 – 178,4 хв (два проходи через приймальне відділення та лабораторія), типу 3 – 130,2 хв (табл. 7). Середній інтервал між прибуттями хворих у лабораторію становить 29,94 хв.

Вузьким місцем є приймальне відділення: завантаження лікарів 0,839, середня черга 3,51 хворих, середнє очікування 47,9 хв (табл. 8). Хворі типу 1 мають пріоритет і чекають значно менше, тому їхній час у системі найменший, а хворі типів 2 і 3 при високому завантаженні лікарів чекають довго. Лабораторія та супровідні завантажені слабо, черги там майже не утворюються.

Для верифікації завантаження кожного пристрою порівняно з балансом потоків: інтенсивність потоку хворих через пристрій, помножена на середній час обслуговування й поділена на кількість каналів. Наприклад, для приймального відділення $(0{,}5\cdot 15 + 0{,}1\cdot 40 + 0{,}4\cdot 30 + 0{,}1\cdot 15) / 15 / 2 = 0{,}8333$ (останній доданок – повторна реєстрація хворих типу 2), для лабораторії інтенсивність прибуття (0,1 +

0,4) / 15, тобто середній інтервал між прибуттями 30 хв. Для всіх пристроїв значення імітації збігаються з балансовими в межах довірчих інтервалів.

Висновок

У роботі реалізовано універсальний алгоритм імітації моделей масового обслуговування з багатоканальними пристроями, пріоритетною чергою та вибором маршруту за пріоритетом, за ймовірністю, за умовою (типом вимоги) або за довільним правилом. Вимоги моделюються окремими об’єктами з атрибутами, що дозволяє обчислювати час перебування та змінювати тип вимоги під час руху моделлю. Правильність алгоритму підтверджено порівнянням з точними розв’язками для систем M/M/c/K, системи з відмовами Ерланга, розщеплення пуассонівського потоку та системи з відносними пріоритетами.

Для банку для автомобілістів складено формалізовану модель з двома одноканальними пристроями, правилом вибору коротшої смуги та правилом перешикування останнього автомобіля. Модель показала, що в годину пік банк справляється з потоком: черги короткі, відмов практично немає, але завантаження касирів нерівномірне (0,41 і 0,20) через перевагу першої смуги, а переїзди між смугами відбуваються в середньому близько 108 разів за 1000 од. часу.

Для лікарні складено формалізовану модель з різнотипними хворими, пріоритетною чергою в приймальному відділенні, умовними маршрутами та поверненням хворих типу 2 після лабораторії. Найбільше часу хворі проводять в очікуванні в приймальному відділенні, завантаження лікарів якого становить близько 0,84; середній час у системі – близько 35 хв для типу 1, 178 хв для типу 2 і 130 хв для типу 3, а інтервал між прибуттями в лабораторію – 30 хв. Обидві моделі побудовано з тих самих класів без зміни алгоритму імітації, що підтверджує його універсальність.